

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Operatori ed espressioni

Espressioni e operatori

Un'**espressione** combina **operandi** tramite **operatori**, e produce un **valore** di un certo **tipo**.

- Esempi di espressioni: `3`, `x`, `x + 1`, `v0 * t - 0.5 * g * t * t`.
- Ogni espressione ha un **tipo**, deciso dai tipi degli operandi (lezione 10).
- Operatori **binari**: agiscono su due operandi (`a + b`).
- Operatori **unari**: agiscono su un solo operando (`-a`, `i++`).
- Anche l'assegnamento `x = 5` è un'espressione: il suo valore è quello assegnato.

```
// anatomia di un'espressione
h = h0 - 0.5 * g * t * t;

■ variabili    ■ costanti    ■ operatori
```

Il computer valuta l'espressione **un operatore alla volta**, in un ordine preciso (argomento di questa lezione).

Operatori aritmetici

Operatore	Significato	Esempio	Risultato
`+`	Somma	`7 + 2`	`9`
`-`	Sottrazione	`7 - 2`	`5`
`*`	Moltiplicazione	`7 * 2`	`14`
`/`	Divisione	`7 / 2`	`3` (interi!)
`%`	Resto (modulo)	`7 % 2`	`1`
`-` (unario)	Cambio di segno	`-x`	

In `C` non esiste l'operatore "potenza"!

```
1   int a = 2 ^ 3;     // a vale 1, non 8!
2   double x2 = x * x; // quadrato: si moltiplica
```

- `^` esiste, ma è un'operazione **sui bit** (XOR): il compilatore non vi darà né errore, né un warning.
- Per le potenze generiche useremo la funzione `pow` della libreria matematica.
- Non esiste nemmeno la moltiplicazione **implicita**: `2x` è un errore, si scrive `2 * x`.

Divisione intera e resto

- Tra **interi**, `/` dà il quoziente troncato e `%` il resto
- Vale sempre la relazione:

$$a = (a/b) \cdot b + a \,\%\, b$$

- Con i numeri negativi il quoziente si tronca **verso lo zero** e il resto ha il **segno del dividendo**.
- `%` funziona **solo tra interi**:

```
1   error: invalid operands to binary % (have ‘double’ and ‘int’)
```

a	b	a / b	a % b
7	2	3	1
-7	2	-3	-1
7	-2	-3	1
6	3	2	0
2	5	0	2

Divisione o resto con divisore `0` tra interi: il programma si interrompe!
(*Floating point exception*).

A cosa serve il resto?

- **Parità:** `n % 2` vale `0` se `n` è pari, `1` se è dispari (per `n ≥ 0`).
- **Divisibilità:** `n % k == 0` se `k` divide `n` (utile per i numeri primi).
- **Cifre di un numero:** `n % 10` è l'ultima cifra, `n / 10 % 10` la penultima.
- **Unità di misura miste:** secondi → ore, minuti, secondi.
- **Fenomeni periodici:** il giorno della settimana, la posizione su un orologio.

```c
#include <stdio.h>

int main() {
    int t = 10000;            // durata [s]

    int ore     = t / 3600;
    int minuti  = t % 3600 / 60;
    int secondi = t % 60;

    printf("%d h %d min %d s\n",
           ore, minuti, secondi);
    return 0;
}
```

```
$ ./tempo
2 h 46 min 40 s
```

Assegnamento composto

Capita spesso di aggiornare una variabile usando il suo valore precedente: il C ha una scrittura abbreviata.

Forma abbreviata	Equivale a
`x += a;`	`x = x + a;`
`x -= a;`	`x = x - a;`
`x *= a;`	`x = x * a;`
`x /= a;`	`x = x / a;`
`n %= k;`	`n = n % k;`

```c
double t = 0.0, dt = 0.01;   // [s]
double somma = 0.0;

t += dt;           // avanza il tempo
somma += 3.5;      // accumula un valore
```

Attenzione

```c
int x = 10;
x *= 2 + 3;        // x = x * (2 + 3) = 50
                   // NON x * 2 + 3 = 23
```

L'espressione a destra viene calcolata **per intero**, poi combinata con `x`.

Incremento e decremento

`++` e `--` aumentano o diminuiscono di **1** una variabile intera: `i++` equivale a `i += 1`.

Prefisso: `++i`

Prima incrementa, poi usa il **nuovo** valore.

```
1   int i = 5;
2   int b = ++i;    // b = 6, i = 6
```

Postfisso: `i++`

Prima usa il **vecchio** valore, poi incrementa.

```
1   int i = 5;
2   int a = i++;    // a = 5, i = 6
```

- Da soli (`i++;` oppure `++i;`) i due modi sono **equivalenti**.
- La differenza conta solo **dentro** un'espressione più grande.
- Li useremo moltissimo nei **cicli** per contare: `for (i = 0; i < n; i++)`

Mai modificare la stessa variabile due volte in un'espressione: `i = i++;` oppure `i++ + i++` hanno comportamento **indefinito**.
`-Wall` vi avvisa: `operation on 'i' may be undefined`.

Precedenza degli operatori

Quale operazione viene eseguita per prima?
Come in matematica, alcuni operatori hanno **precedenza** su altri.

Precedenza	Operatori	Associatività
alta	`x++` `x--` (postfissi)	→
	`++x` `--x` `-x` `(tipo)` `sizeof`	←
	`*` `/` `%`	→
	`+` `-`	→
bassa	`=` `+=` `-=` `*=` `/=` `%=`	←

Tabella ridotta agli operatori visti finora. La versione completa è nel libro (§3.3).

- `*`, `/` e `%` vengono **prima** di `+` e `-`: `2 + 3 * 4` vale `14`, non `20`.
- Il **cast** viene prima della divisione: `(double) a / b` converte solo `a` (lezione 10).
- L'**assegnamento** viene per ultimo: prima si calcola tutto il lato destro.

Associatività

Se più operatori hanno la **stessa precedenza**, decide l'**associatività**.

Da sinistra a destra →

La maggior parte degli operatori binari:

```
10 - 4 - 3    // (10 - 4) - 3 = 3
8 / 4 / 2     // (8 / 4) / 2  = 1
```

Da destra a sinistra ←

Assegnamenti e operatori unari:

```
a = b = 0;    // a = (b = 0): entrambe valgono 0
```

Tipi e ordine di valutazione

L'ordine conta anche per le **conversioni**:

```
double r1 = 3 / 2 * 2.0;   // 2.0
double r2 = 2.0 * 3 / 2;   // 3.0
```

- Matematicamente uguali, numericamente **diversi**!

Le parentesi

Le parentesi tonde `( )` forzano l'ordine di valutazione: **nel dubbio, usatele**.

Formula	In C
$\frac{a+b}{2}$	`(a + b) / 2`
$\frac{a}{b\,c}$	`a / (b * c)`
$v = \frac{x_2 - x_1}{t_2 - t_1}$	`(x2 - x1) / (t2 - t1)`
$E = \frac{1}{2}mv^2$	`0.5 * m * v * v`

- La **frazione** della matematica nasconde delle parentesi: numeratore e denominatore vanno racchiusi.
- `a / b * c` **non** è $\frac{a}{bc}$ ma $\frac{a}{b} \cdot c$, per l'associatività da sinistra.
- Esistono solo le parentesi **tonde**: niente quadre o graffe nelle espressioni (hanno un significato ben preciso in `C`)
- Parentesi in più non costano nulla e rendono il codice più **leggibile**.

Mettiamo tutto insieme: moto accelerato

```c
#include <stdio.h>

int main() {
    const double a = 2.0;     // [m/s^2]
    double x0 = 1.0;          // [m]
    double v0 = 3.0;          // [m/s]
    double t  = 0.0;          // [s]
    double dt = 0.5;          // [s]
    double x;

    t += dt;
    x = x0 + v0 * t + 0.5 * a * t * t;
    printf("t = %f s, x = %f m\n", t, x);

    return 0;
}
```

```
$ ./moto
t = 0.500000 s, x = 2.750000 m
```

- `t += dt` aggiorna il tempo.
- Precedenza: prima i prodotti, poi le somme. Nessuna parentesi necessaria.
- `0.5` e non `1/2` !
- **Test:**

  $x = 1 + 3 \cdot 0.5 + \frac{1}{2} \cdot 2 \cdot 0.25 = 2.75$ ✓
- Non sarebbe male poter ripetere il passo `t += dt` molte volte…

Mettetevi alla prova

Quanto valgono queste espressioni? Pensateci **prima** di andare avanti.

```
1   int    a = 17 % 5;
2   int    b = -17 % 5;
3   int    c = 2 + 3 * 4 - 6 / 4;
4   int    d = 10 / 3 * 3;
5   double e = 1 / 2 + 1 / 2.0;
6   int    f = 3 ^ 2;
7   int    i = 3;
8   int    g = i++ * 2;    // e i?
```

Variabile	Valore	Perché
`a`	`2`	$17 = 3 \cdot 5 + 2$
`b`	`-2`	Il resto ha il segno del dividendo
`c`	`13`	`2 + 12 - 1`: `6 / 4` vale `1`
`d`	`9`	`(10 / 3) * 3 = 3 * 3`
`e`	`0.5`	`0 + 0.5`
`f`	`1`	`^` non è la potenza!
`g`	`6`	Postfisso: usa `3`, poi `i` diventa `4`